

第二章 语法分析

(自顶向下的分析技术)

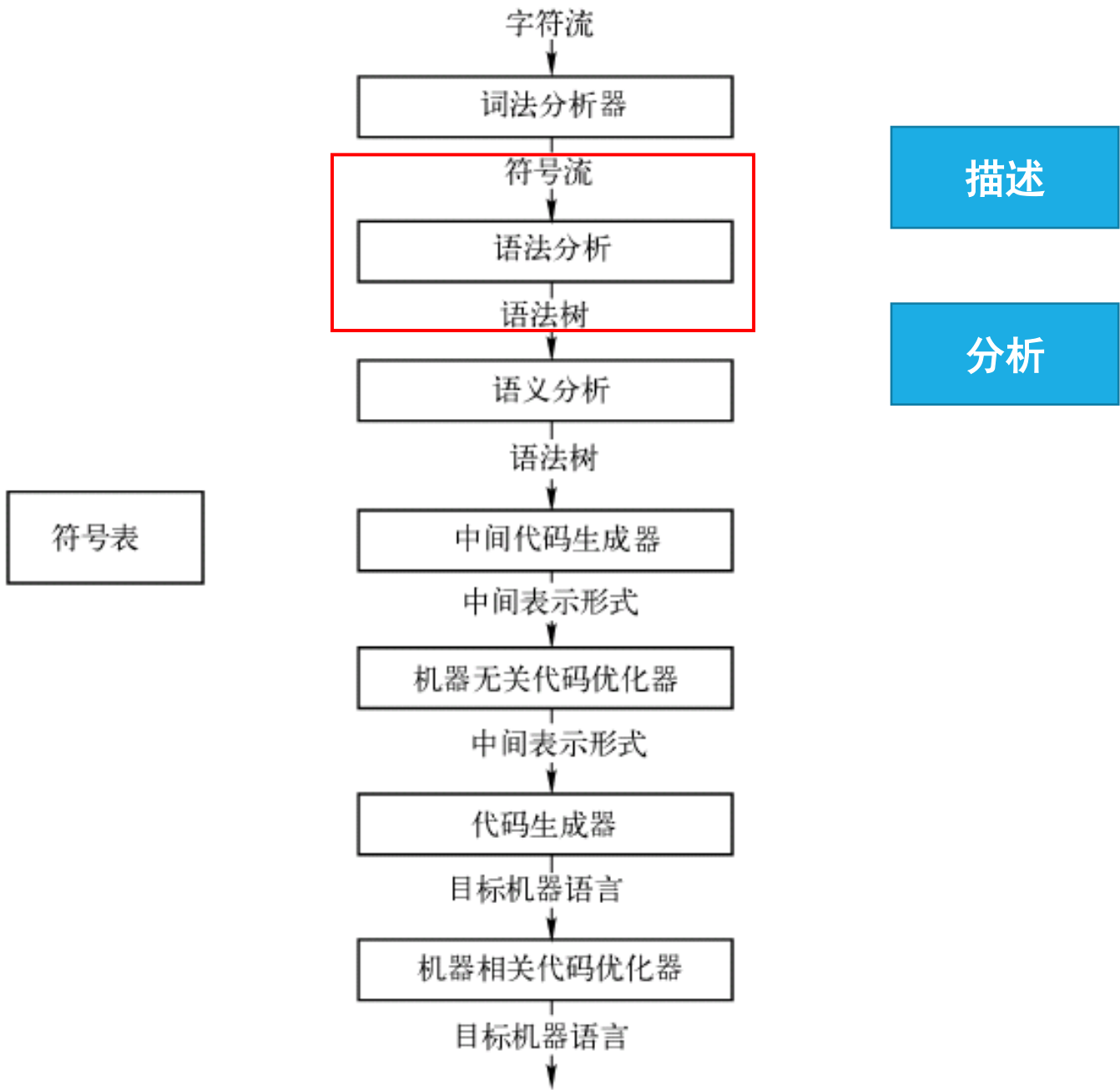
冯洋

南京大学计算机学院

fengyang@nju.edu.cn

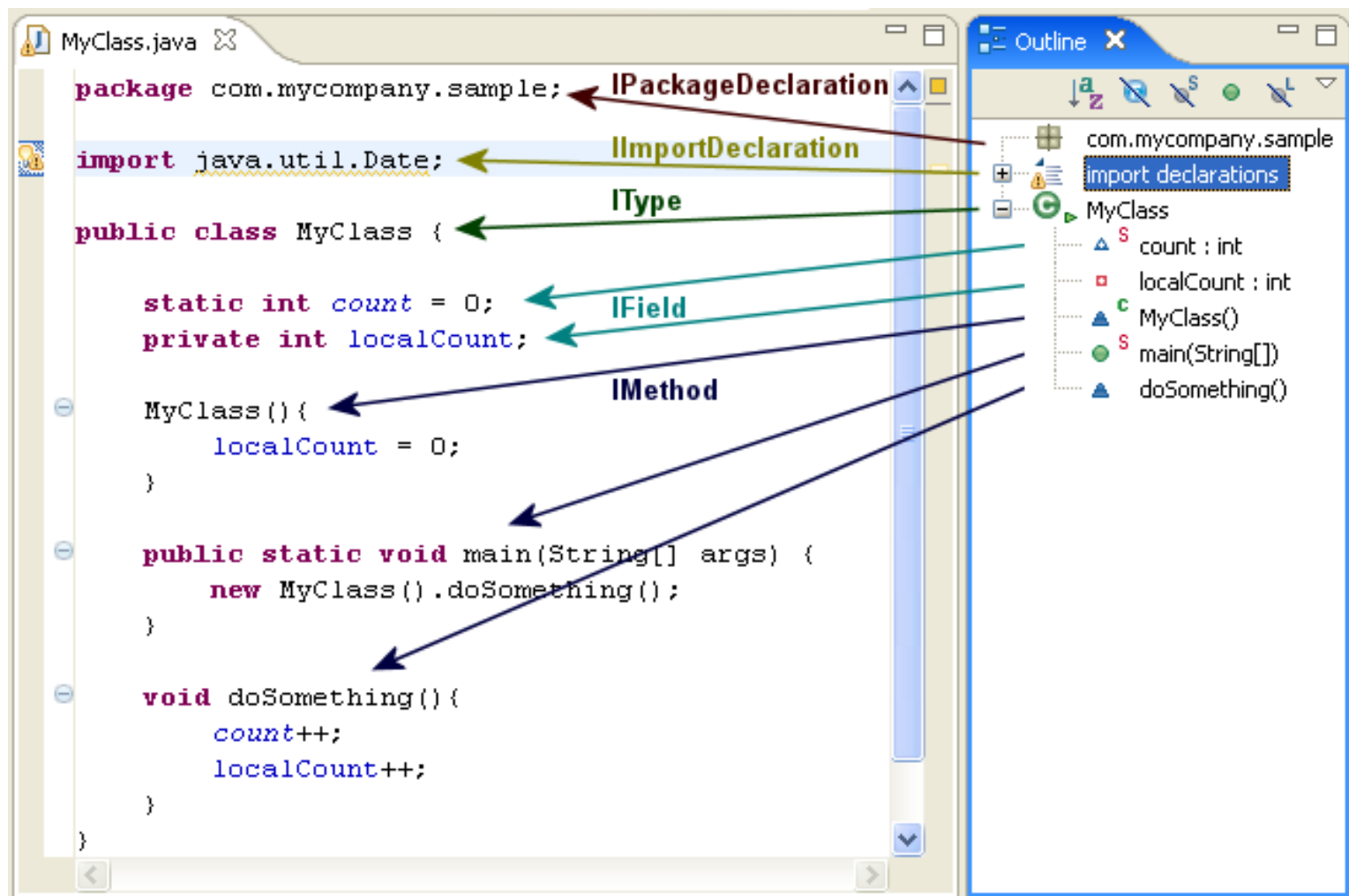
提纲

- 上下文无关文法
- 语法分析技术
 - 自顶向下
 - 自底向上
- 语法分析器生成工具



引言

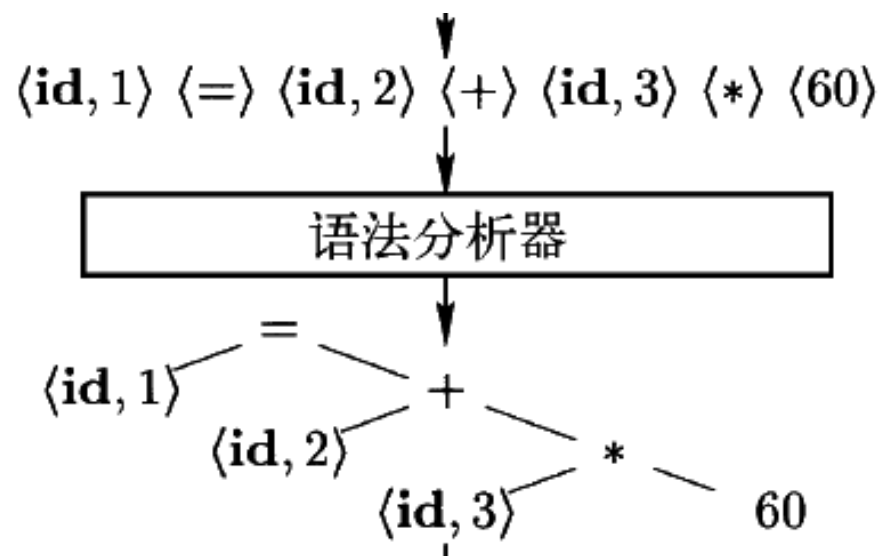
- 计算机程序是由很多语法结构构成



引言

position = initial + rate * 60

<id,1> <=, > <id, 2> <+, > <id,3> <*, > <number, 4>

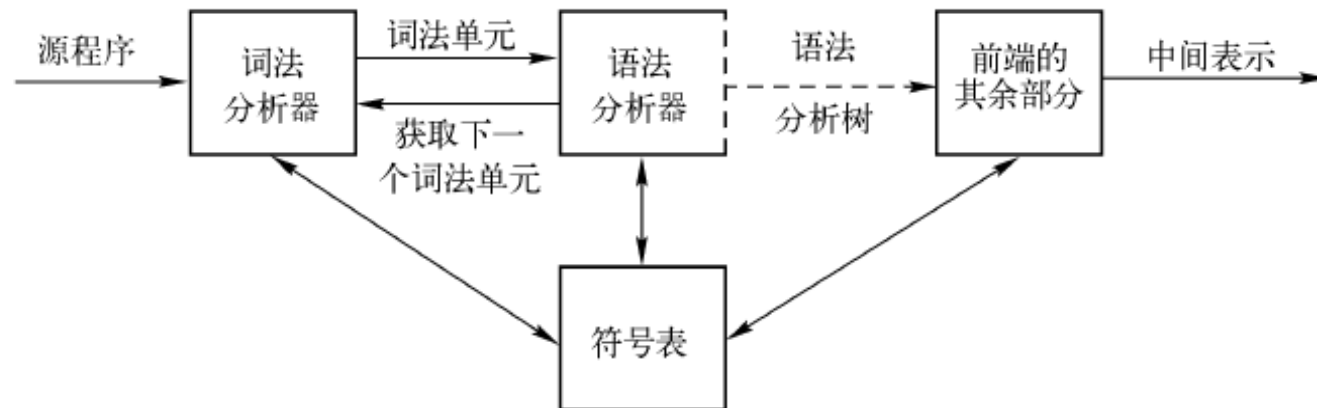


语法分析

引言

• 语法分析器

- 输入：词法分析器输出的词法单元序列
- 输出：语法树表示
- 语法分析器功能：
 - 验证输入源程序的合法性，输出良构程序的语法结构
 - 对于病构的程序，能够报告语法错误，进行错误恢复
 - 类型检查，语义分析，翻译生成中间代码等往往和语法分析过程**交错完成**，实践中往往和语法分析放入一个模块，图上用“前端的其余部分”表示上述活动
- 语法分析器的类型：
 - 通用型
 - 自顶向下：通常处理**LL**文法
 - 自底向上：通常处理**LR**文法



上下文无关文法

- 语法 (Syntax): 语法是描述一种程序设计语言中结构规则的总称, 即哪些符号组合是合法的, 哪些是不合法的
 - 直观理解: 语法回答的问题是——“一段程序在形式上是否符合语言的规范?”
 - 例子:
 - `if (x > 0) y = y + 1;` 在 C 语言中符合语法
 - `if x > 0) { y = y + 1;` 则语法错误 (括号不配对)
- 文法 (Grammar) 定义: 文法是一种形式化工具, 通常是上下文无关文法 (CFG), 用来精确刻画语言的语法规则。
 - 直观理解: 文法是用产生式规则形式化写下来的“语法定义”
 - 例子:
 - $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \text{other}$
 - $E \rightarrow \text{id} > \text{id}$

上下文无关文法

维度	语法 (Syntax)	文法 (Grammar)
定义	程序设计语言中结构规则的总称	形式化工具（常用上下文无关文法）来精确刻画语法规则
层次	概念层面，描述语言整体的合法结构	工具层面，语法的形式化表示
表现方式	自然语言描述、规范说明	产生式规则（如 BNF/EBNF）
示例	“if 语句必须有匹配的括号”	$S \rightarrow \text{if } E \text{ then } S \text{ else } S$
作用	判断源程序是否形式正确	构造语法分析器、生成语法树/AST 的依据
类比	自然语言的“语法知识”	语法书中的“形式化规则”

上下文无关文法

- 定义：上下文无关文法（**Context Free Grammar, CFG**）是一种四元组形式的形式文法，通常记作

$$G=(V,\Sigma,P,S)$$

- 其中：

- V 为非终结符（变量）的有限集合
- Σ 为终结符的有限集合，且 $V \cap \Sigma = \emptyset$
- $S \in V$ 为开始符号
- P 为产生式的有限集合，每个产生式形如 $A \rightarrow \alpha, A \in V, \alpha \in (V \cup \Sigma)^*$

- 在 CFG 中，每条产生式的左部只能是单个非终结符，而右部可以是任意长度的终结符与非终结符串

上下文无关文法

- 关于定义的两个问题：

- 什么是上下文？

- 为什么说是“上下文无关”？

上下文无关文法

- 关于定义的两个问题：

- 什么是上下文？

- 在应用一个产生式进行推导时，前后已经推导出的部分结果就是上下文
 - 非终结符出现时的“左右邻居”
 - 在文法推导中，一个非终结符 A 出现的位置通常是在某个字符串里，比如：
 - “ $\alpha A \beta$ ” 这里， α 是 A 左边的符号串， β 是 A 右边的符号串
 - 这两个部分 α 和 β 就构成了 A 出现时的上下文 (context)

- 为什么说“上下文无关”？

- 上下文无关的意思的，只要文法的定义里有某个产生式，不管一个非终结符前后的串是什么，就可以应用相应的产生式进行推导
 - 推导（替换）过程不依赖左右邻居

上下文无关文法

- 上下文无关文法 (CFG) 的特点

- 在 CFG 中，产生式是 $A \rightarrow \gamma$
- 应用规则时，完全不考虑 A 左右有什么符号，只要看到一个 A，就可以替换
- 因此称为“上下文无关”：替换规则只取决于 A 本身，不取决于它的上下文 α 、 β
- 例子：对于产生式规则 $S \rightarrow aSb \mid \epsilon$ ，在任何位置，只要遇到 S，都可以用 aSb 或 ϵ 替换

上下文无关文法

- 上下文有关文法 (CSG) 的特点

- 在 CSG 中，产生式一般形如： $\alpha A \beta \rightarrow \alpha \gamma \beta$
- 应用规则时，这意味着：只有当 A 处于特定上下文 α 、 β 之中时，才能被替换为 γ
- 因此称为“上下文有关”： A 的替换依赖于它的周围环境
- 例子：对于产生式规则 $aB \rightarrow ab$ ，这里 B 只有在左边是 a 的情况下，才能替换为 b ；如果 B 出现在别的地方，就不行

上下文无关文法

- 上下文有关文法 (CSG) 的特点

- 在 CSG 中，产生式一般形如： $\alpha A \beta \rightarrow \alpha \gamma \beta$
- 应用规则时，这意味着：只有当 A 处于特定上下文 α 、 β 之中时，才能被替换为 γ
- 因此称为“上下文有关”： A 的替换依赖于它的周围环境
- 例子：对于产生式规则 $aB \rightarrow ab$ ，这里 B 只有在左边是 a 的情况下，才能替换为 b ；如果 B 出现在别的地方，就不行

上下文无关文法

项目	上下文无关文法 (CFG)	上下文有关文法 (CSG)
产生式形式	$A \rightarrow \alpha A$, 左部必须是单个非终结符	$\alpha A \beta \rightarrow \alpha \gamma \beta$, 左部可含上下文
依赖性	替换只依赖于非终结符自身	替换依赖于非终结符的左右上下文
典型语言	平衡括号、正则表达式扩展、编程语言的大多数语法结构	$\{a^n b^n c^n \mid n \geq 1\}$ 某些语义约束相关语言
计算能力	等价于下推自动机 (PDA)	等价于线性有界自动机 (LBA)

上下文无关文法

- 代表性文法示例

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

LR文法类
自底向上分析

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

LL类文法
(无左递归)
自顶向下分析

上下文无关文法

• 代表性文法示例

Identifier:
IDENTIFIER

QualifiedIdentifier:
Identifier { . Identifier }

QualifiedIdentifierList:
QualifiedIdentifier { , QualifiedIdentifier }

CompilationUnit:
[[Annotations] package QualifiedIdentifier ;]
{ImportDeclaration} {TypeDeclaration}

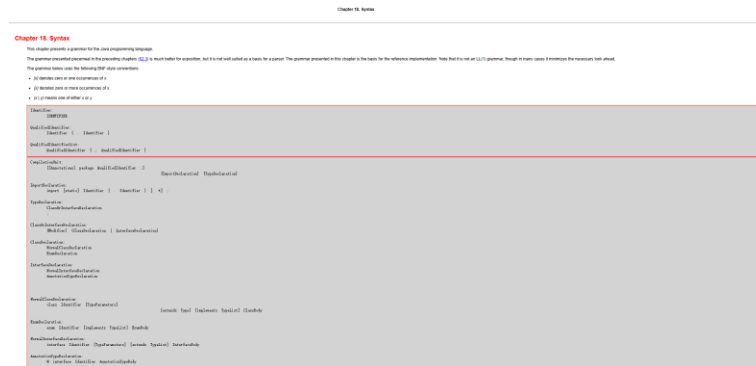
ImportDeclaration:
import [static] Identifier { . Identifier } [. *] ;

TypeDeclaration:
ClassOrInterfaceDeclaration
;

ClassOrInterfaceDeclaration:
{Modifier} (ClassDeclaration | InterfaceDeclaration)

ClassDeclaration:
NormalClassDeclaration
EnumDeclaration

InterfaceDeclaration:
NormalInterfaceDeclaration
AnnotationTypeDeclaration



NormalClassDeclaration:
class Identifier [TypeParameters]
[extends Type] [implements TypeList] ClassBody

EnumDeclaration:
enum Identifier [implements TypeList] EnumBody

NormalInterfaceDeclaration:
interface Identifier [TypeParameters] [extends TypeList] InterfaceBody

AnnotationTypeDeclaration:
@ interface Identifier AnnotationTypeBody

上下文无关文法

• 代表性文法示例

```

BlockExpression →
    {
        InnerAttribute*
        Statements?
    }

Statements →
    Statement+
    | Statement+ ExpressionWithoutBlock
    | ExpressionWithoutBlock

AsyncBlockExpression → async move? BlockExpression

ConstBlockExpression → const BlockExpression

UnsafeBlockExpression → unsafe BlockExpression

OperatorExpression →
    BorrowExpression
    | DereferenceExpression
    | ErrorPropagationExpression
    | NegationExpression
    | ArithmeticOrLogicalExpression
    | ComparisonExpression
    | LazyBooleanExpression
    | TypeCastExpression
    | AssignmentExpression
    | CompoundAssignmentExpression

BorrowExpression →
    ( & | && ) Expression
    | ( & | && ) mut Expression
    | ( & | && ) raw const Expression
    | ( & | && ) raw mut Expression

DereferenceExpression → * Expression

ErrorPropagationExpression → Expression ?

```

```

NegationExpression →
    - Expression
    | ! Expression

ArithmeticOrLogicalExpression →
    Expression + Expression
    | Expression - Expression
    | Expression * Expression
    | Expression / Expression
    | Expression % Expression
    | Expression & Expression
    | Expression | Expression
    | Expression ^ Expression
    | Expression << Expression
    | Expression >> Expression

ComparisonExpression →
    Expression == Expression
    | Expression != Expression
    | Expression > Expression
    | Expression < Expression
    | Expression >= Expression
    | Expression <= Expression

LazyBooleanExpression →
    Expression || Expression
    | Expression && Expression

TypeCastExpression → Expression as TypeNoBounds

AssignmentExpression → Expression = Expression

CompoundAssignmentExpression →
    Expression += Expression
    | Expression -= Expression
    | Expression *= Expression
    | Expression /= Expression
    | Expression %= Expression
    | Expression &= Expression
    | Expression |= Expression
    | Expression ^= Expression
    | Expression <<= Expression
    | Expression >>= Expression

GroupedExpression → ( Expression )

ArrayExpression → [ ArrayElements? ]

ArrayElements →
    Expression ( , Expression )* ,?
    | Expression ; Expression

```

上下文无关文法

- 符号表示约定

- 终结符号: $a\ b\ +\ 3\ \mathbf{id}\dots$
- 非终结符号: $A\ B\ C\dots, S, \mathit{stmt}$
- 文法符号: $X\ Y\ \dots$
- 终结符号串: $u\ v\ w\dots$
- 文法符号串: $\alpha\ \beta\dots$
- 不同可选体: $\alpha_1\ \alpha_2\ \alpha_3\dots$
- 开始符号: S

语法分析技术基础

- 推导

- 将待处理的串中的某个非终结符号替换为这个非终结符号的某个产生式的体
- 从开始符号出发，不断进行上面的替换，就可以得到文法的不同句型

- 产生式又可称为**重写规则**（Rewriting rule）

- 推导的实例

- 文法： $E \rightarrow -E \mid E+E \mid E * E \mid (E) \mid id$
- 从E到 $-(id)$ 的推导序列： $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(id)$

语法分析技术基础

- 推导的一般性定义

➤ 如果 $A \rightarrow \gamma$ 是一个产生式, 那么 $\alpha A \beta \Rightarrow \alpha \gamma \beta$

- 经过零步或者多步推导出: $\Rightarrow^*$

➤ 对于任何串 α , $\alpha \Rightarrow^* \alpha$

➤ 如果 $\alpha \Rightarrow^* \beta$ 且 $\beta \Rightarrow \gamma$, 那么 $\alpha \Rightarrow^* \gamma$

- 经过一步或者多步推导出: $\Rightarrow^+$

➤ $\alpha \Rightarrow^+ \beta$ 且 α 不等于 β 等价于 $\alpha \Rightarrow^* \beta$

- 最左(右)推导

□ 符号: $\xRightarrow{*}_{lm}$ $\xRightarrow{*}_{rm}$

语法分析技术基础

- 文法推导实例

$$E \rightarrow E+E \mid E^*E \mid -E \mid (E) \mid \mathbf{id}$$

从上述文法推导出串 $-(\mathbf{id}+\mathbf{id})$

$$E \Rightarrow - \quad E \Rightarrow - (E) \Rightarrow - (E + E) \Rightarrow - (\mathbf{id} + E) \Rightarrow - (\mathbf{id} + \mathbf{id})$$

语法分析技术基础

- 句型/句子/语言

- 句型 (sentential form) :

- 如果 $S \xRightarrow{*} \alpha$, 那么 α 就是文法的一个句型

- 可能既包含非终结符, 又包含终结符号; 可以是空串

- 句子 (sentence)

- 文法的句子就是不包含非终结符号的句型

- 语言

- 文法 G 的语言就是 G 的句子的集合, 记为 $L(G)$

- ω 在 $L(G)$ 中当且仅当 ω 是 G 的句子, 即 $S \xRightarrow{*} \omega$

语法分析技术基础

- 从推导的角度看，语法分析的任务是：接受一个终结符号串作为输入，
找出从文法的开始符号推导出这个串的方法
- 从推导的角度看，核心问题是：
 - 选择哪个非终结符进行替换
 - 选择哪个产生式进行替换

$$E \rightarrow E+E \mid E * E \mid -E \mid (E) \mid id$$

推导出串 $-(id+id*id)$

语法分析技术基础

- 通常使用两种方式进行推导
 - 最左推导：总是选择每个句型的最左非终结符号。记作 $\Rightarrow_{lm}$
 - 最右推导：总是选择最右边的非终结符号。记作 $\Rightarrow_{rm}$
- 每个最左推导步骤可以写成 $wA\gamma \xRightarrow{lm} w\delta\gamma$
 - 应用产生式： $A \rightarrow \delta$
- 如果 α 经过最左推导得到 β ，记作 $\alpha \xRightarrow{*}_{lm} \beta$
- 最左句型：S 是文法 G 的识别符号，如果 $S \xRightarrow{*}_{lm} \alpha$ 则 α 是 G 的最左句型

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

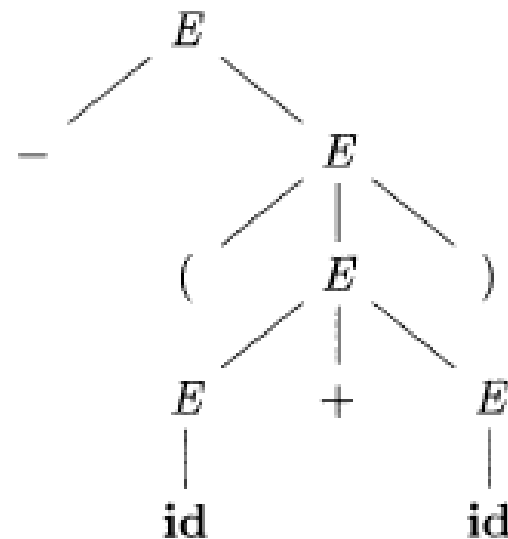
$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \mathbf{id}) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

语法分析技术基础

- 语法分析树，一种推导的图形表示形式
 - 根结点的标号是文法的开始符号
 - 每个叶子结点的标号是非终结符号、终结符号或 ϵ
 - 每个内部节点的标号是非终结符号
 - 每个内部结点表示某个产生式的一次应用
 - 内部结点的标号为产生式头，结点的子结点从左到右是产生式的体
- 有时允许树的根不是开始符号（对应于某个短语）
- 树的叶子组成的序列是根的文法符号的句型
- 一棵分析树可对应多个推导序列，但是分析树和最左（右）推导序列之间具有一一对应关系

语法分析技术基础

- 语法分析树是推导的图形表示形式，树上看不出来推导的顺序
- 能够反映串的语法层次结构
- 语法分析树
 - 内部节点：对应于一个非终结符号
 - 子节点：对应于其父节点为头的产生式体
 - 叶子节点：可以是终结符号或非终结符号，从左到右排列可以得到一个句型，称为这棵树的结果



- (id + id) 的语法分析树

$E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \text{id}) \Rightarrow -(\text{id} + \text{id})$

语法分析技术基础

- 假设有推导序列：

$$\triangleright A \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$$

- 算法：

- 初始化： α_1 的分析树是标号为A的单个结点

- 假设已经构造出 $\alpha_{i-1} = X_1 X_2 \cdots X_k$ 的分析树，且 α_{i-1} 到 α_i 的推导是将 X_j 替换为 β ，那么在当前分析树中找出第j个非 ε 结点，向这个结点增加构成 β 的子结点。（如果 $\beta = \varepsilon$ ，则增加一个标号为 ε 的子结点）

$$E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$$

$$E \Rightarrow - \quad E \Rightarrow - (E) \Rightarrow - (E + E) \Rightarrow - (\text{id} + E) \Rightarrow - (\text{id} + \text{id})$$

语法分析技术基础

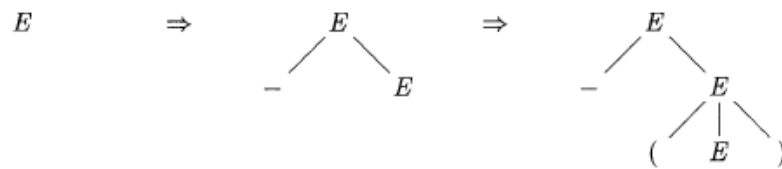
- 假设有推导序列:

$$\triangleright A \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$$

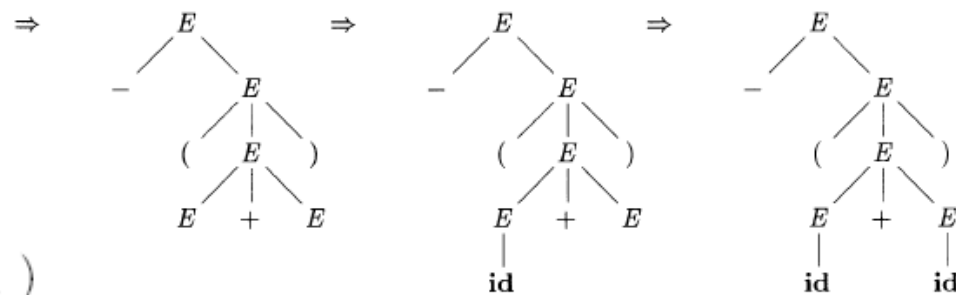
- 算法:

- 初始化: α_1 的分析树是标号为 A 的单个结点

- 假设已经构造出 $\alpha_{i-1} = X_1 X_2 \dots X_k$ 的分析树, 且 α_{i-1} 到 α_i 的推导是将 X_j 替换为 β , 那么在当前分析树中找出第 j 个非 ε 结点, 向这个结点增加构成 β 的子结点。(如果 $\beta = \varepsilon$, 则增加一个标号为 ε 的子结点)



$$E \Rightarrow E + E \mid E * E \mid -E \mid (E) \mid \mathbf{id}$$



$$E \Rightarrow - \quad E \Rightarrow - (E) \Rightarrow - (E + E) \Rightarrow - (\mathbf{id} + E) \Rightarrow - (\mathbf{id} + \mathbf{id})$$

语法分析技术基础

- 在进行**高效**的语法分析之前，需要对文法做以下处理
 - 消除二义性
 - 文法的二义性：文法可以为一个句子生成多颗不同的树
 - 消除左递归
 - 左递归：文法中一个非终结符号A使得对某个串 α ，存在一个推导 $A \xRightarrow{+} A\alpha$ ，则称这个文法是左递归的
 - 提取左公因子

语法分析技术基础

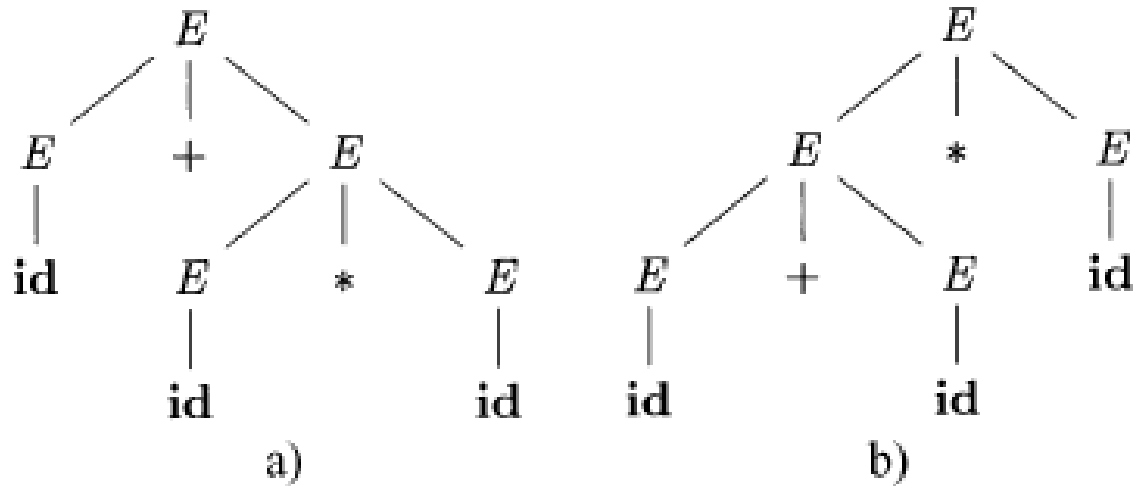
• 文法的二义性：

➤ 定义：如果一个文法可以为一个给定的句子生成多棵不同的语法分析树，则该文法为二义性文法(ambiguous grammar)

➤ 例子：

□ 语法规则： $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$

□ 给定串： $\text{id} + \text{id} * \text{id}$



语法分析技术基础

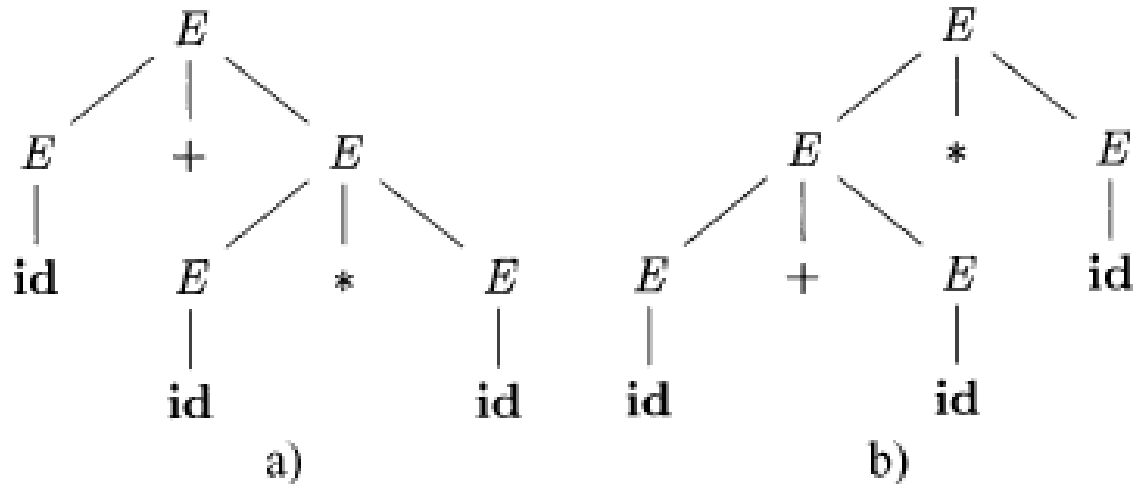
• 文法的二义性：

➤ 定义：如果一个文法可以为一个给定的句子生成多棵不同的语法分析树，则该文法为二义性文法(ambiguous grammar)

➤ 例子：

□ 语法规则： $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$

□ 给定串： $\text{id} + \text{id} * \text{id}$



通常情况下，我们需要无二义性的文法

语法分析技术基础

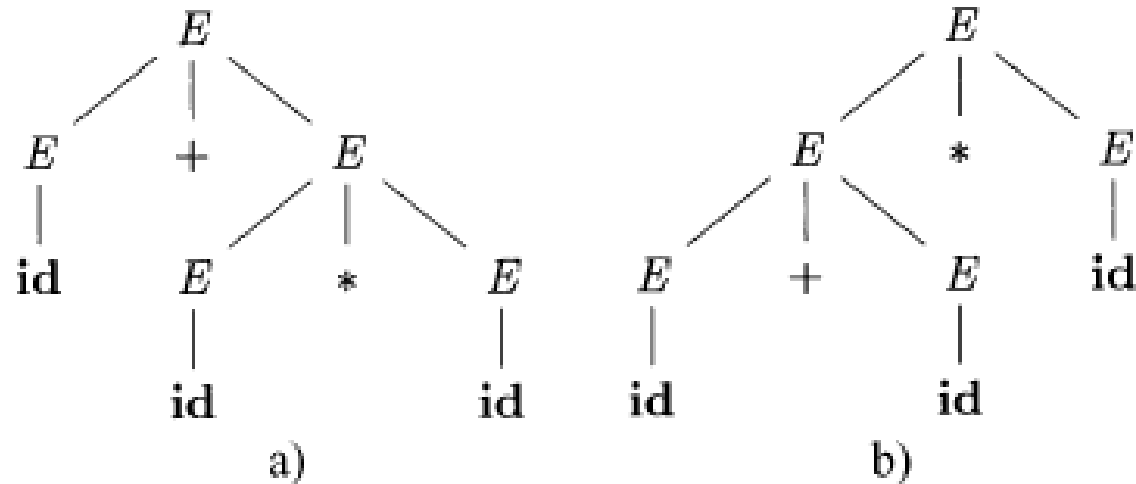
• 文法的二义性：

➤ 定义：如果一个文法可以为一个给定的句子生成多棵不同的语法分析树，则该文法为二义性文法(ambiguous grammar)

➤ 例子：

□ 语法规则： $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$

□ 给定串： $\text{id} + \text{id} * \text{id}$



通常情况下，我们需要无二义性的文法，但是.....

语法分析技术基础

- 文法的二义性：
- 在实际的文法定义中，二义性却有一些潜在的“好处”
 - 简洁性：
 - 二义性文法往往比等价的无二义文法更短、更容易书写。例如表达式文法若强行消除二义性，会引入大量非终结符层次，影响可读性。
 - 贴近语言直观：
 - 一些二义性（如算术表达式）本身就是人类自然语言的习惯，通过附加优先级规则即可直观解决。
 - 工具生成灵活性：
 - 对于手工编写编译器，使用二义性文法+消解规则（precedence/associativity declaration）往往更容易实现，而不是强行改写文法。

语法分析技术基础

- 文法的二义性：

- 例子：

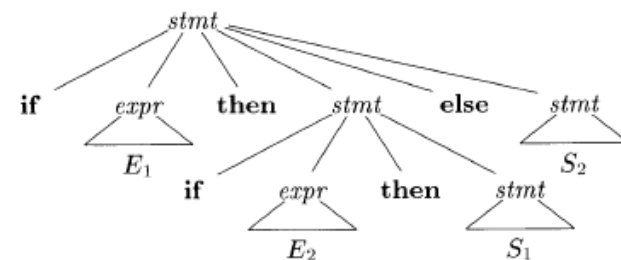
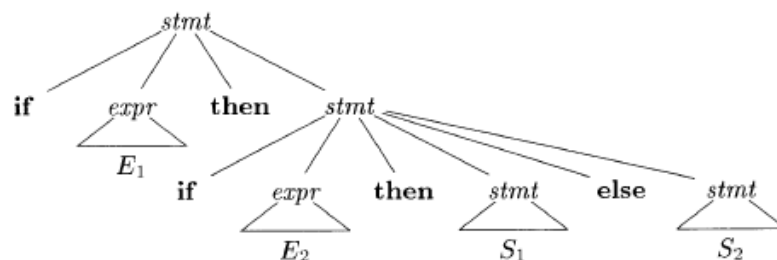
➤ C / C++ 的 “dangling else” 问题

$\text{Stmt} \rightarrow \text{if Expr then Stmt}$
 $\quad | \text{if Expr then Stmt else Stmt}$
 $\quad | \text{other}$

```

if (x > 0)
    if (y > 0)
        print("A");
    else
        print("B");
    
```

思考，右边这个代码的输出是什么？



语法分析技术基础

- 文法的二义性：

- 例子：

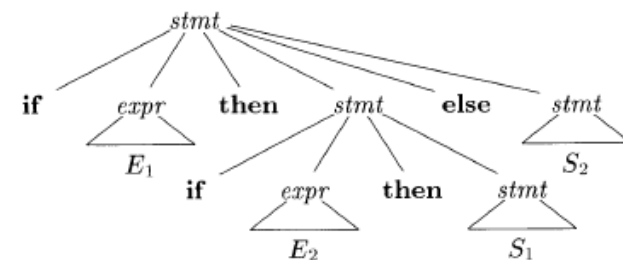
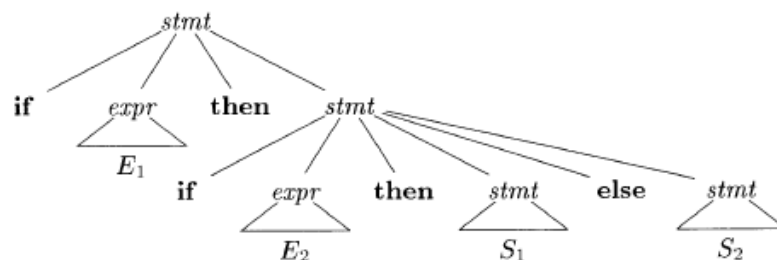
➤ C / C++ 的 “dangling else” 问题

$\text{Stmt} \rightarrow \text{if Expr then Stmt}$
 $\quad | \text{if Expr then Stmt else Stmt}$
 $\quad | \text{other}$

```

if (x > 0)
    if (y > 0)
        print("A");
    else
        print("B");
    
```

思考，右边这个代码的输出是什么？



- 当 $x > 0, y > 0$ 时，输出 **A**
- 当 $x > 0, y \leq 0$ 时，输出 **B**
- 当 $x \leq 0$ 时，无输出

语法分析技术基础

- 文法的二义性：
- 例子：
 - C / C++ 的 “dangling else” 问题
 - C++ 模板语法的 `<` 和 `>>`
 - 在早期 C++ 中，`vector<vector<int>> v;` 会被误读为移位运算符 `>>`，造成二义性。C++11 之后语言标准修改了解析规则，将其视为右尖括号闭合。

语法分析技术基础

- 文法的二义性消解的方法

- 语法层面消解

- 重写文法，使得句子只有一种推导方式

- 例：表达式文法通过引入 Term、Factor 等非终结符实现

- 语义规则消解

- 在保持二义性文法的同时，增加优先级 / 结合性 / 绑定约定来选择正确解析

- 常见于算术运算符、布尔表达式。

- 上下文信息消解

- 利用符号表或语义分析阶段的信息来决定解析方式

- C 的 typedef / identifier 问题即属于此类

- 语言标准约定

- 直接在语言规范中写明二义性情况下的处理方式

- 例如悬空 else 总是归最近的 if

语法分析技术基础

- **Recap:**语言是由文法的开始符号出发，能够推导得到的所有句子的集合
 - 文法G: $S \rightarrow aS|a|b$, $L(G)=\{a^i(a|b), i \geq 0\}$
 - 文法G: $S \rightarrow aSb|ab$, $L(G)=\{a^n b^n, n \geq 1\}$
 - 文法G: $S \rightarrow (S)S|\epsilon$, $L(G)=\{\text{所有具有对称括号对的串}\}$

语法分析技术基础

阶段	输入	输出	描述体系
词法分析	源程序符号串	词法单元序列	正则表达式
句法分析	词法单元序列	语法树	上下文无关文法

- 文法比正则表达式描述能力更强
- 正则表达式描述词法单元比较简洁
- 基于正则表达式构造的词法分析器效率更高
- 正则表达式适合描述词法结构，文法适合描述嵌套结构

语法分析技术基础

- 上下文无关文法的表达能力更强
- 每个正则表达式都可以用一个上下文无关文法来描述，反之不成立
- 每个正则语言都是一个上下文无关语言，反之不成立
- 正则表达式 $(a|b)^*abb$ 等价于文法

$$\begin{aligned}A_0 &\rightarrow a A_0 \mid b A_0 \mid a A_1 \\A_1 &\rightarrow b A_2 \\A_2 &\rightarrow b A_3 \\A_3 &\rightarrow \varepsilon\end{aligned}$$

语法分析技术基础

- 左递归（Left Recursion）：

- 定义：文法中一个非终结符号 A 使得对某个串 α ，存在一个推导 $A \xRightarrow{+} A\alpha$ ，则称这个文法是左递归的

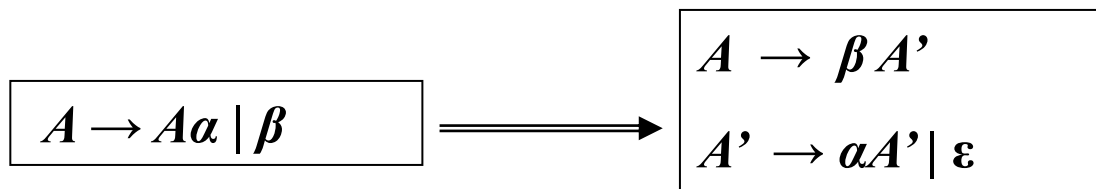
- 立即左递归

- 如果存在 $A \rightarrow A\alpha$ ，则称为**立即**左递归，注意其中 A 是非终结符， α 是符号串（可能为空）， β 是不以 A 开头的符号串

- 为什么要消除左递归？

- 自顶向下的语法分析技术不能处理左递归的文法，如 LL(1) 分析器），左递归会导致无限递归

- 立即左递归的消除实例：



语法分析技术基础

- 左递归（Left Recursion）：
- 假设非终结符号A存在立即左递归的情形，假设以A为左部的规则有：

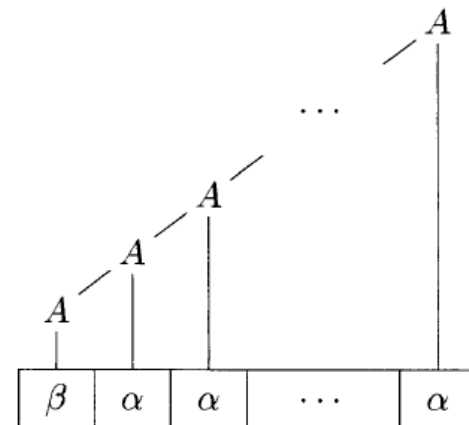
$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

首先对A产生式**分组**（所有 α_i 不等于 ϵ ， β_i 不以A开头）
可以替换为

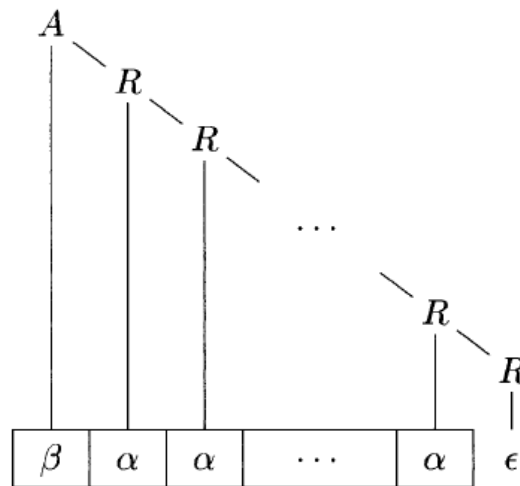
$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon$$

- 由A生成的串总是以某个 β_i 开头，然后跟上零个或者多个 α_i 的重复



a)



b)

语法分析技术基础

- 左递归 (Left Recursion)

➤ $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

➤ 分组

$A \rightarrow \boxed{A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m} \mid \boxed{\beta_1 \mid \beta_2 \mid \dots \mid \beta_n}$

➤ 替换

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$

语法分析技术基础

- 立即左递归消除示例

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F \quad \Longrightarrow$$

$$F \rightarrow (E) \mid \mathbf{id}$$

语法分析技术基础

- 立即左递归消除示例

$$\begin{array}{l}
 E \rightarrow E + T \mid T \\
 T \rightarrow T * F \mid F \\
 F \rightarrow (E) \mid \mathbf{id}
 \end{array}
 \Longrightarrow
 \begin{array}{l}
 E \rightarrow TE' \\
 E' \rightarrow + TE' \mid \epsilon \\
 T \rightarrow FT' \\
 T' \rightarrow * FT' \mid \epsilon \\
 F \rightarrow (E) \mid \mathbf{id}
 \end{array}$$

语法分析技术基础

- 间接左递归（多步左递归）
- 消除立即左递归的方法并不能消除因为两步或多步推导而产生的左递归
- 文法：
$$S \rightarrow Aa \mid b,$$
$$A \rightarrow Ac \mid Sd \mid \varepsilon$$
$$S \Rightarrow Aa \Rightarrow Sda$$
- 如何消除？

语法分析技术基础

- 输入：没有环 $A_i \Rightarrow^+ A_i$ 和 $A \rightarrow \varepsilon$
- 输出：一个等价的无左递归的文法
- 算法原理：
 - 给非终结符号**排序**， $A_1, A_2, \dots, A_n$
 - 如果只有 $A_i \Rightarrow A_j$ ($i < j$)，则不会有左递归
 - 如果发现 $A_i \Rightarrow A_j$ ($i > j$)，**代入** A_j 的当前产生式，若替换后有 A_i 的直接左递归，再**消除**

语法分析技术基础

- 输入：没有环和 ϵ 产生式的文法G
- 输出：等价的无左递归的文法
- 步骤：

将文法的非终结符号任意**排序**为 $A_1, A_2, \dots, A_n$

for i=1 to n do {

 for j = 1 to i-1 do

 {

 将形如 $A_i \rightarrow A_j \gamma$ 的产生式**替换**为 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,

 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是以 A_j 为左部的所有产生式;

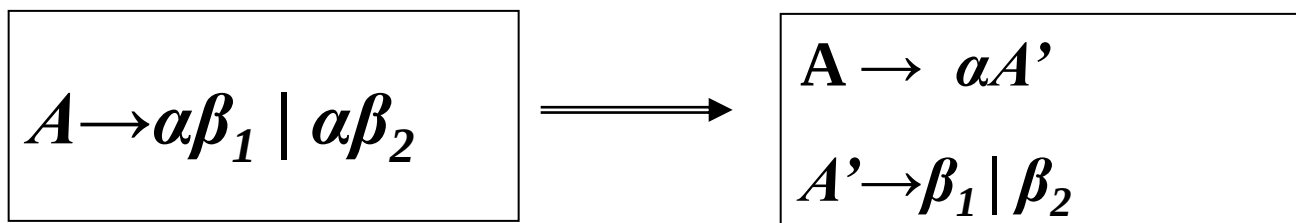
 }

消除 A_i 的立即左递归;

}

语法分析技术基础

- 提取左公因子
- 在构造 自顶向下预测分析器（如 LL(1) 分析）时，分析器需要根据下一个输入符号来唯一决定使用哪一个产生式，如果有多个产生式中下一个符号相同，则不知道该如何选择（自顶向下算法会详细描述）

$$\begin{aligned}
 stmt \rightarrow & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | \textbf{ if } expr \textbf{ then } stmt
 \end{aligned}$$


语法分析技术基础

- 输入：文法 G
- 输出：一个等价的提取了左公因子的文法
- 方法：对于每个非终结符号 A ，找出它的两个或多个可选项之间的最长公共前缀 α ，且 $\alpha \neq \varepsilon$ ，那么将 A 所有的产生式

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$

替换为

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

语法分析技术基础

- 输入：没有环和 ϵ 产生式的文法 G
- 输出：等价的无左递归的文法
- 步骤：

将文法的非终结符号任意**排序**为 $A_1, A_2, \dots, A_n$

for $i=1$ to n do {

 for $j = 1$ to $i-1$ do

 {

 将形如 $A_i \rightarrow A_j \gamma$ 的产生式**替换**为 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,

 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是以 A_j 为左部的所有产生式;

 }

消除 A_i 的立即左递归;

}

语法分析技术基础

- 提取左公因子

$$\text{Stmt} \rightarrow \text{if Expr then Stmt else Stmt} \\ | \text{if Expr then Stmt}$$
$$\text{Stmt} \rightarrow \text{if Expr then Stmt Stmt'} \\ \text{Stmt'} \rightarrow \text{else Stmt} \mid \varepsilon$$

自顶向下分析技术

- 自顶向下分析可以被看作是为输入串构造语法分析树的问题，也可以看作是一个寻找输入串的**最左推导**的过程
- 问题：在推导的每一步，对非终结符号A，**应用哪个产生式**，以可能产生于输入串相匹配的终结符号串

自顶向下分析技术

id+id*id的自顶向下分析

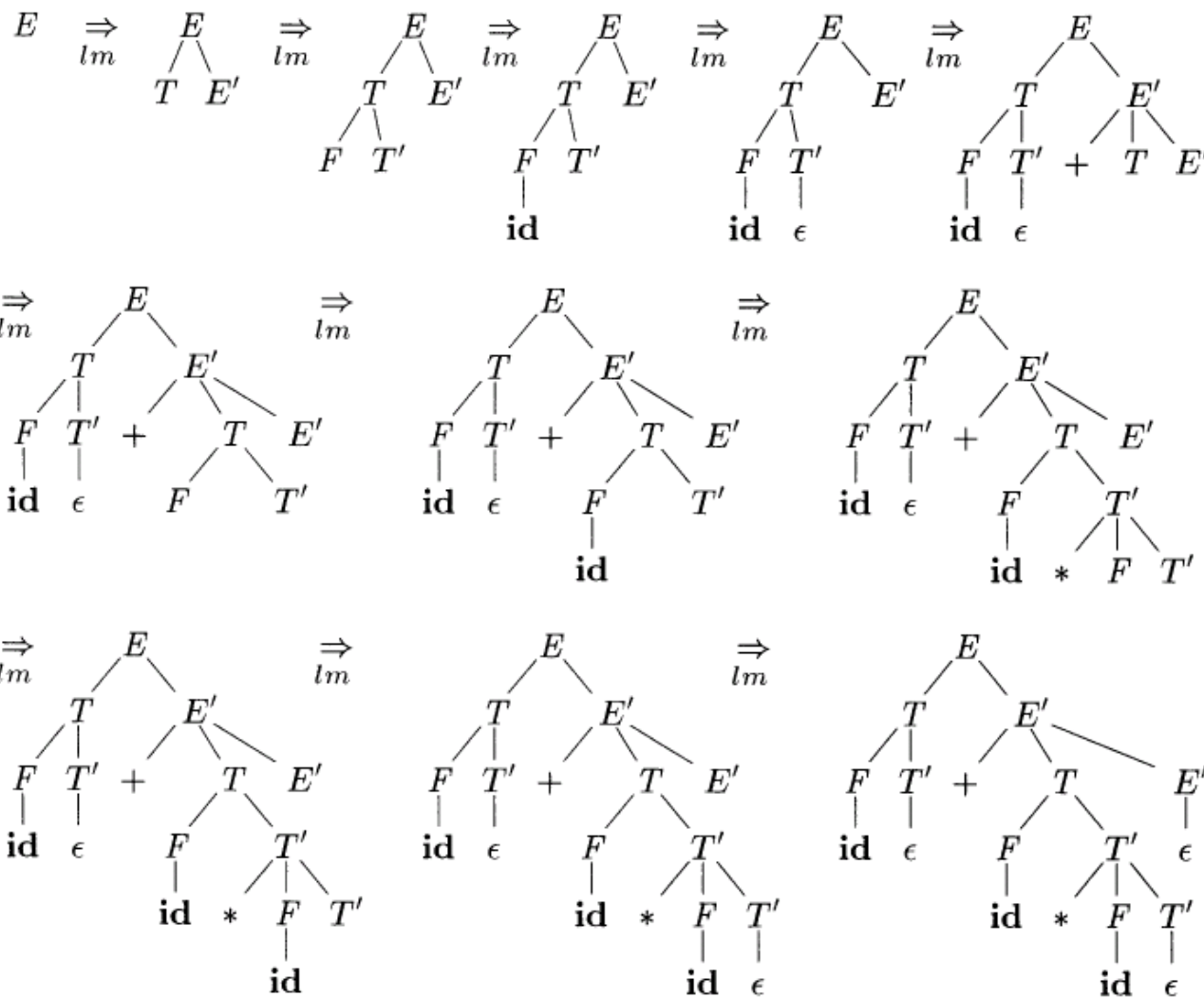
$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$



自顶向下分析技术

- 问题：对于非终结符号 A ，选择哪一个产生式
- 一种**通用**的**递归下降**分析框架
 - 由一组过程组成，每个非终结符号对应一个过程（函数）
 - 程序的执行从开始符号对应的过程开始
 - 每个过程的功能是：选择一个产生式体，扫描相应的句子。若遇到非终结符号，调用该符号对应的过程
 - 输入流逐步匹配文法产生式，构造语法树

```

void A() {
1)      选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)      for (  $i = 1$  to  $k$  ) {
3)          if (  $X_i$  是一个非终结符号 )
4)              调用过程  $X_i()$ ;
5)          else if (  $X_i$  等于当前的输入符号  $a$  )
6)              读入下一个输入符号;
7)          else /* 发生了一个错误 */;
      }
}
  
```


自顶向下分析技术

- **递归下降**分析框架的核心思想:

- 每个非终结符 $\rightarrow$ 一个过程/函数

- 如 $\text{Expr} \rightarrow \text{Term} \{ + \text{Term} \}$

- 定义 `parseExpr()`, 内部调用 `parseTerm()`

- 自顶向下展开

- 从文法的开始符号开始

- 逐步递归调用子过程, 直到匹配输入

- 错误处理

- 一旦输入符号与期望不符 $\rightarrow$ 报错/回溯

```

void A() {
1)     选择一个 A 产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)     for ( i = 1 to k ) {
3)         if (  $X_i$  是一个非终结符号 )
4)             调用过程  $X_i()$ ;
5)         else if (  $X_i$  等于当前的输入符号  $a$  )
6)             读入下一个输入符号;
7)         else /* 发生了一个错误 */;
    }
}
    
```

自顶向下分析技术

- 递归下降分析框架的代码示例:

$E \rightarrow E + T$
 $T \rightarrow \text{id}$

```
void A() {  
1)      选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
2)      for (  $i = 1$  to  $k$  ) {  
3)          if (  $X_i$  是一个非终结符号 )  
4)              调用过程  $X_i()$ ;  
5)      else if (  $X_i$  等于当前的输入符号  $a$  )  
6)          读入下一个输入符号;  
7)      else /* 发生了一个错误 */;  
        }  
}
```

自顶向下分析技术

- 递归下降分析框架的代码示例：

$E \rightarrow E + T$
 $T \rightarrow \text{id}$

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \varepsilon$
 $T \rightarrow \text{id}$

```
void A() {  
1)    选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  是一个非终结符号 )  
4)            调用过程  $X_i()$ ;  
5)        else if (  $X_i$  等于当前的输入符号  $a$  )  
6)            读入下一个输入符号;  
7)        else /* 发生了一个错误 */;  
    }  
}
```

自顶向下分析技术

• 递归下降分析框架的代码示例：

$$\begin{array}{l} E \rightarrow E + T \\ T \rightarrow \text{id} \end{array} \qquad \begin{array}{l} E \rightarrow T E' \\ E' \rightarrow + T E' \mid \varepsilon \\ T \rightarrow \text{id} \end{array}$$

```
procedure parseE():
  parseT()
  parseEPrime()
```

```
procedure parseEPrime():
  if next_token == '+':
    match('+')
    parseT()
    parseEPrime()
  else:
    return //  $\varepsilon$ 
```

```
void A() {
1)   选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)   for (  $i = 1$  to  $k$  ) {
3)       if (  $X_i$  是一个非终结符号 )
4)           调用过程  $X_i()$ ;
5)       else if (  $X_i$  等于当前的输入符号  $a$  )
6)           读入下一个输入符号;
7)       else /* 发生了一个错误 */;
    }
}
```

自顶向下分析技术

• 递归下降分析框架的代码示例：

$$\begin{array}{l} E \rightarrow E + T \\ T \rightarrow \text{id} \end{array} \qquad \begin{array}{l} E \rightarrow T E' \\ E' \rightarrow + T E' \mid \varepsilon \\ T \rightarrow \text{id} \end{array}$$

```
procedure parseE():
  parseT()
  parseEPrime()
```

```
procedure parseEPrime():
  if next_token == '+':
    match('+')
    parseT()
    parseEPrime()
  else:
    return //  $\varepsilon$ 
```

```
void A() {
1)   选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)   for (  $i = 1$  to  $k$  ) {
3)       if (  $X_i$  是一个非终结符号 )
4)           调用过程  $X_i()$ ;
5)       else if (  $X_i$  等于当前的输入符号  $a$  )
6)           读入下一个输入符号;
7)       else /* 发生了一个错误 */;
    }
}
```

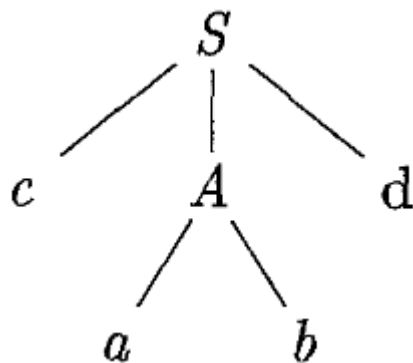
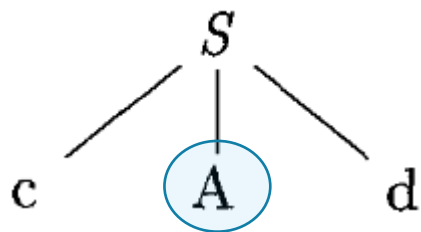
自顶向下分析技术

• 输入串 $w = c a d$

• 文法:

➤ $S \rightarrow cAd$

➤ $A \rightarrow ab \mid a$



```
void A() {
1)   选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)   for (  $i = 1$  to  $k$  ) {
3)       if (  $X_i$  是一个非终结符号 )
4)           调用过程  $X_i()$ ;
5)       else if (  $X_i$  等于当前的输入符号  $a$  )
6)           读入下一个输入符号;
7)       else /* 发生了一个错误 */;
    }
}
```

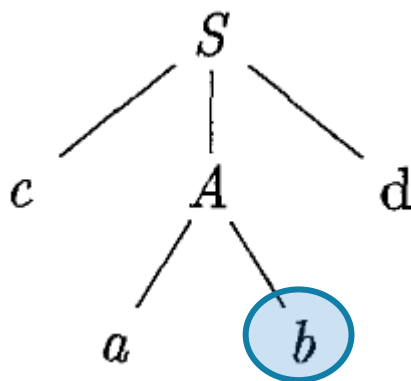
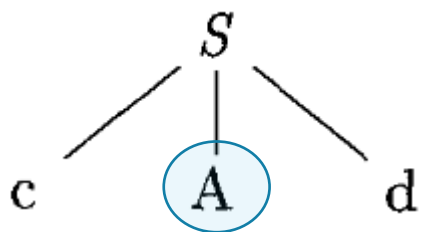
自顶向下分析技术

• 输入串 $w = c a d$

• 文法:

➤ $S \rightarrow cAd$

➤ $A \rightarrow ab \mid a$



```
void A() {
1)   选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)   for (  $i = 1$  to  $k$  ) {
3)       if (  $X_i$  是一个非终结符号 )
4)           调用过程  $X_i()$ ;
5)       else if (  $X_i$  等于当前的输入符号  $a$  )
6)           读入下一个输入符号;
7)       else /* 发生了一个错误 */;
    }
}
```

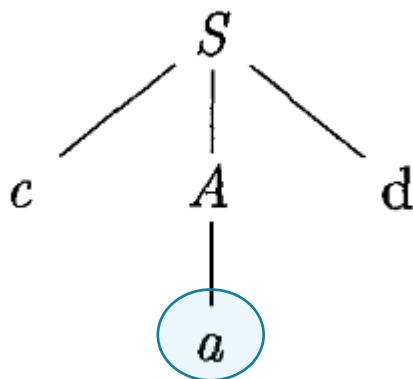
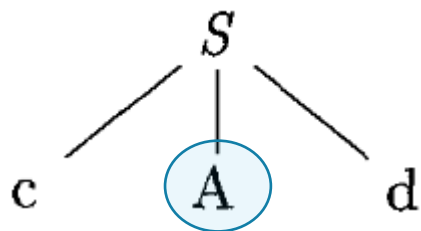
自顶向下分析技术

• 输入串 $w = c a d$

• 文法：

➤ $S \rightarrow cAd$

➤ $A \rightarrow ab \mid a$



```
void A() {
1)  选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)  for (  $i = 1$  to  $k$  ) {
3)      if (  $X_i$  是一个非终结符号 )
4)          调用过程  $X_i()$ ;
5)      else if (  $X_i$  等于当前的输入符号  $a$  )
6)          读入下一个输入符号;
7)      else /* 发生了一个错误 */;
    }
}
```

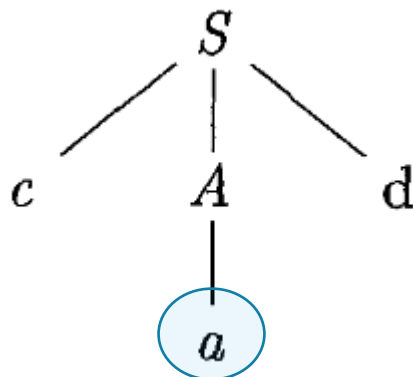
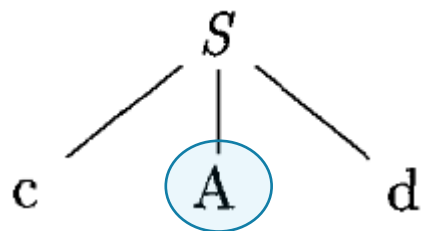

自顶向下分析技术

• 输入串 $w = c a d$

• 文法:

➤ $S \rightarrow cAd$

➤ $A \rightarrow ab \mid a$



```
void A() {
1)  选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)  for (  $i = 1$  to  $k$  ) {
3)      if (  $X_i$  是一个非终结符号 )
4)          调用过程  $X_i()$ ;
5)      else if (  $X_i$  等于当前的输入符号  $a$  )
6)          读入下一个输入符号;
7)      else /* 发生了一个错误 */;
    }
}
```

匹配成功!

自顶向下分析技术

- 可能需要回溯，或者说，可能需要重复扫描输入
- “在回到A时，我们必须把输入指针重新设置到位置2，即我们第一次尝试展开A时该指针指向的位置。这意味着A的过程必须将输入指针存放在一个局部变量中。”
- 如果文法中存在左递归.....
- 回溯（穷试 ☺）显然是笨办法
- **解决思路**
 - 如果在分析时能“预测”该选择哪条产生式，就能避免回溯
 - 这就引出了 预测分析法（**Predictive Parsing**）

自顶向下分析技术

- 预测分析法 是一种**无回溯**的自顶向下语法分析
- 核心：根据当前输入符号和预测表（Parsing Table），唯一确定使用哪条产生式
 - 输入符号流 + 语法规则 → 分析过程
 - 前瞻符号 (Lookahead Symbol)：只看一个输入符号，就能决定使用哪条产生式
- 基本要求：
 - 文法必须是 LL(1) 文法
 - 有多个可能的产生式时预测分析法无能为力
 - 即：在任意一个非终结符的选择分支中，前瞻符号能唯一确定选择的产生式

自顶向下分析技术

- 预测分析法 是一种**无回溯**的自顶向下语法分析
- 核心：根据当前输入符号和预测表（Parsing Table），唯一确定使用哪条产生式
 - 输入符号流 + 语法规则 → 分析过程
 - 前瞻符号 (Lookahead Symbol)：只看一个输入符号，就能决定使用哪条产生式
- 基本要求：
 - 文法必须是 LL(1) 文法
 - 有多个可能的产生式时预测分析法无能为力
 - 即：在任意一个非终结符的选择分支中，前瞻符号能唯一确定选择的产生式

第一个L：从左到右扫描输入（Left-to-right）

第二个L：最左推导（Leftmost derivation）

1：只需要一个前瞻符号 Lookahead

自顶向下分析技术

- 预测分析法
- 比如文法： $E \rightarrow +EE \mid -EE \mid id$ ；输入为 **+ id - id id**
- 当两个产生式具有相同的前缀时无法预测
文法： $stmt \rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \mid \text{if } expr \text{ then } stmt$
输入： **if a then ...**
改写后文法： $stmt \rightarrow \text{if } expr \text{ then } stmt \text{ else } Part$
 $elsePart \rightarrow \text{else } stmt \mid \varepsilon$
- 需要提取公因子

自顶向下分析技术

- 预测分析法
- 一种**确定性的**、**无回溯**的分析技术
- 在每一步选择正确的产生式

□ 例1: 文法G(S):

$S \rightarrow pA$

$S \rightarrow qB$

$A \rightarrow cAd$

$A \rightarrow a$

□ 输入串 $W = pccadd$

□ 例2: 文法G(S)为:

$S \rightarrow Ap$

$S \rightarrow Bq$

$A \rightarrow a \mid cA$

$B \rightarrow b \mid dB$

□ 输入 $W = ccap$

自顶向下分析技术

- 预测分析法基本处理框架

- 预处理文法

- 消除左递归

- 提取左公因子

- 转换为 LL(1) 文法

- 构造预测分析表（Parsing Table）

- 行：非终结符

- 列：终结符（含 \$）

- 表项：应使用的产生式

- 分析过程

- 使用一个栈保存符号

- 从栈顶符号和输入符号查表，决定推导步骤

- 直到栈空、输入符号读完为止

自顶向下分析技术

- 构造预测分析表（Parsing Table）
- 预测分析（LL(1)）要“看一个前瞻符号就能选唯一产生式”
 - 工具：
 - FIRST 用来判断候选式首终结符能是什么；
 - FOLLOW 用来在候选式可推出 ϵ 时，决定“空产生式”在何处可用
 - 目标：构造解析表 $M[A, a]$ —— 让“栈顶非终结符 A + 当前输入 a ”唯一确定产生式

自顶向下分析技术

- 构造预测分析表（Parsing Table）

- 对符号串 $\alpha \in (V \cup \Sigma)^*$:

特例:

$\text{FIRST}(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow a \cdots\} \cup (\{\epsilon\} \text{ 若 } \alpha \Rightarrow^* \epsilon)$ 终结符 a : $\text{FIRST}(a) = \{a\}$
空串 ϵ : $\text{FIRST}(\epsilon) = \{\epsilon\}$

- 递推规则

- $\text{FIRST}(a) = \{a\}$ (a 为终结符) ; $\text{FIRST}(\epsilon) = \{\epsilon\}$
- $\text{FIRST}(A)$ (A 为非终结符) : 对每个产生式 $A \rightarrow \alpha$, 把 $\text{FIRST}(\alpha) - \{\epsilon\}$ 加入 $\text{FIRST}(A)$; 若 $\alpha \Rightarrow^* \epsilon$, 再把 ϵ 加入
- 字串首符号传播: 对于 $\text{FIRST}(X_1 X_2 \cdots X_k)$ 的构造
 - 先加 $\text{FIRST}(X_1) - \{\epsilon\}$
 - 若 $\epsilon \in \text{FIRST}(X_1)$, 再加 $\text{FIRST}(X_2) - \{\epsilon\}$
 - 依此类推; 若所有 X_i 都可推出 ϵ , 则加入 ϵ

自顶向下分析技术

- 构造预测分析表 (Parsing Table)

- 对非终结符 A

$$\text{FOLLOW}(A) = \{a \in \Sigma \mid \exists S \Rightarrow^* \alpha A a \beta\} \cup (\{\$ \} \text{ 若 } A \text{ 能出现在句子末尾})$$

直观：能紧跟在 A 之后出现的终结符集合；起始符号 S 的 FOLLOW 集含输入结束标记 $\$$

- 递推规则

- 将 $\$$ 加入 $\text{FOLLOW}(S)$ (S 为开始符号)

- 对每个产生式 $A \rightarrow \alpha B \beta$:

- 把 $\text{FIRST}(\beta) - \{\epsilon\}$ 加入 $\text{FOLLOW}(B)$

- 若 $\epsilon \in \text{FIRST}(\beta)$ 或 $\beta = \epsilon$, 把 $\text{FOLLOW}(A)$ 加入 $\text{FOLLOW}(B)$

自顶向下分析技术

- 构造预测分析表构建示例

- 求FIRST

- 终结符与 ϵ

□ $\text{FIRST}() = \{(\text{), FIRST}() = \{\}$, $\text{FIRST}(\text{id}) = \{\text{id}\}$, $\text{FIRST}(+) = \{+\}$, $\text{FIRST}(*) = \{*\}$, $\text{FIRST}(\epsilon) = \{\epsilon\}$

➤ $F \rightarrow (E) \mid \text{id} \Rightarrow \text{FIRST}(F) = \{(\text{, id}\}$

➤ $T' \rightarrow *FT' \mid \epsilon \Rightarrow \text{FIRST}(T') = \{*, \epsilon\}$

➤ $T \rightarrow FT' \Rightarrow \text{FIRST}(T) = \text{FIRST}(F) = \{(\text{, id}\} \quad (\text{因 } \epsilon \notin \text{FIRST}(F))$

➤ $E' \rightarrow +TE' \mid \epsilon \Rightarrow \text{FIRST}(E') = \{+, \epsilon\}$

➤ $E \rightarrow TE' \Rightarrow \text{FIRST}(E) = \text{FIRST}(T) = \{(\text{, id}\}$

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

自顶向下分析技术

• 构造预测分析表构建示例

• 求Follow

- 初始化: FOLLOW(E)={ $\$$ }, 其余为空
- $F \rightarrow (E) \mid id \Rightarrow FOLLOW(E) = \{), \$\}$
- $E \rightarrow T E' \Rightarrow FOLLOW(T) = \{+,), \$\}, FOLLOW(E') = \{), \$\}$
 - 给 FOLLOW(T) 加 FIRST(E') - $\{\epsilon\} = \{+\}$
 - 因 E' 可推出 ϵ , 再把 FOLLOW(E) = $\{), \$\}$ 加到 FOLLOW(T)
 - 末尾的 E' 获得 FOLLOW(E)
- $E' \rightarrow + T E'$ 以及 $T \rightarrow F T' \Rightarrow FOLLOW(F) = \{*, +,), \$\}, FOLLOW(T') = \{+,), \$\}$
 - T 之后是 E', 加 FIRST(E') \ $\{\epsilon\} = \{+\}$ 到 FOLLOW(T) (已在);
 - 因 $E' \Rightarrow^* \epsilon$, 再把 FOLLOW(E') = $\{), \$\}$ 加到 FOLLOW(T) (已在)
 - 给 F 加 FIRST(T') - $\{\epsilon\} = \{*\}$
 - 因 $T' \Rightarrow^* \epsilon$, 再把 FOLLOW(T) = $\{+,), \$\}$ 加到 FOLLOW(F)
 - 末尾的 T' 获得 FOLLOW(T)

$$\begin{aligned}
 E &\rightarrow T E' \\
 E' &\rightarrow + T E' \mid \epsilon \\
 T &\rightarrow F T' \\
 T' &\rightarrow * F T' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

自顶向下分析技术

- 在 LL(1) 分析表中，每个产生式（即文法规则右部）需要决定：当输入符号是什么时，应该选择这条产生式
- 为此，我们定义：
 - 对于一个产生式 $A \rightarrow \alpha$ ，它的 SELECT 集合（记作 $\text{SELECT}(A \rightarrow \alpha)$ ）就是：在什么情况下，预测分析表会用这条产生式。
- 更具体地：
 - 如果 α 能推出某个串，它的第一个符号是终结符 t ，那么 $t \in \text{SELECT}(A \rightarrow \alpha)$ ，即 $\text{FIRST}(\alpha)$
 - 如果 α 可以推出空串 ϵ ，那么 $\text{FOLLOW}(A) \subseteq \text{SELECT}(A \rightarrow \alpha)$ 。（因为此时要看 A 后面能跟什么符号，来决定用这条产生式）
 - 公式总结：

$$\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha) \cup (\text{FOLLOW}(A) \text{ if } \epsilon \in \text{FIRST}(\alpha))$$

自顶向下分析技术

- 为什么要用SELECT集合？
- 在构造预测分析表 $M[A, a]$ 时：
 - 对于每一条产生式 $A \rightarrow \alpha$ ，对于所有 $a \in \text{SELECT}(A \rightarrow \alpha)$ ，填入 $M[A, a] = A \rightarrow \alpha$
 - 换句话说，SELECT 集合决定了“当下一个输入符号是 a 时，要不要选择这条产生式”

$$\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha) \cup (\text{FOLLOW}(A) \text{ if } \epsilon \in \text{FIRST}(\alpha))$$

自顶向下分析技术

- 为什么要用SELECT集合？
- LL(1) 的判别条件：
 - 文法是 LL(1) 的条件之一就是：对于同一非终结符 A 的不同产生式，它们的 **SELECT 集合必须两两不相交**
 - 这样，当下一个输入符号是 a 时，表格里只有唯一一条产生式，没有冲突

如果 SELECT 集合有交集 $\Rightarrow$ 预测表冲突 $\Rightarrow$ 不是 LL(1)

自顶向下分析技术

- 用 FIRST/FOLLOW 构造 LL(1) 预测表
 - 对每个产生式 $A \rightarrow \alpha$:
 - 对所有 $a \in \text{FIRST}(\alpha) - \{\epsilon\}$: 填入 $M[A, a] = A \rightarrow \alpha$
 - 若 $\epsilon \in \text{FIRST}(\alpha)$, 则对所有 $b \in \text{FOLLOW}(A)$: 填入 $M[A, b] = A \rightarrow \alpha$
 - 若某表项出现冲突（同一格两条以上产生式），文法非 LL(1)

自顶向下分析技术

• 用 FIRST/FOLLOW 构造 LL(1) 预测表

- 对每个产生式 $A \rightarrow \alpha A$:
- 对所有 $a \in \text{FIRST}(\alpha) - \{\epsilon\}$: 填入 $M[A, a] = A \rightarrow \alpha$
- 若 $\epsilon \in \text{FIRST}(\alpha)$, 则对所有 $b \in \text{FOLLOW}(A)$: 填入 $M[A, b] = A \rightarrow \alpha$
- 若某表项出现冲突 (同一格两条以上产生式), 文法非 LL(1)
- 对于 LL(1) 文法, 预测表中每个条目都唯一地指定了一个产生式, 或者标明 Error

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

自顶向下分析技术

- 练习题:

考虑文法G:

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow a \mid b \quad (2)$$

$$B \rightarrow Bbb \mid a \quad (3)$$

- (1) 请消除文法G 中的左递归(记新文法为G');
- (2) 请为文法G' 计算必要的First 集合与Follow 集合;
- (3) 请为文法G' 构造预测分析表;
- (4) 请问文法G' 是否是LL(1) 文法, 并说明理由。

自顶向下分析技术

- 练习题:

(1)请消除文法G 中的左递归(记新文法为G');

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow a \mid b \quad (2)$$

$$B \rightarrow Bbb \mid a \quad (3)$$

G':

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow a \mid b \quad (2)$$

$$B \rightarrow aB' \quad (3)$$

$$B' \rightarrow bbB' \mid \epsilon \quad (4)$$

自顶向下分析技术

- 练习题:

(2) 请为文法 G' 计算必要的First 集合与Follow 集合;

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow a \mid b \quad (2)$$

$$B \rightarrow Bbb \mid a \quad (3)$$

$$\text{FIRST}(S) = \{a, b\}$$

$$\text{FIRST}(A) = \{a, b\}$$

$$\text{FIRST}(B) = \{a\}$$

$$\text{FIRST}(B') = \{b, \epsilon\}$$

$$\text{FOLLOW}(S) = \{\$, \}$$

$$\text{FOLLOW}(A) = \text{FIRST}(B) = \{a\},$$

$$\text{FOLLOW}(B) = \text{FOLLOW}(S) = \{\$, \}$$

$$\text{FOLLOW}(B') = \text{FOLLOW}(B) - \{\phi\} = \{\$, \}$$

自顶向下分析技术

• 练习题:

(3) 请为文法 G' 构造预测分析表;

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow a \mid b \quad (2)$$

$$B \rightarrow Bbb \mid a \quad (3)$$

	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	
A	$A \rightarrow a$	$A \rightarrow b$	
B	$B \rightarrow aB'$		
B'		$B' \rightarrow bbB'$	$B' \rightarrow \epsilon$

自顶向下分析技术

- 一个有意思的问题：这个文法能否被 LL(1) 直接处理？如何处理？

$$S \rightarrow a S a$$
$$S \rightarrow a a$$

自顶向下分析技术

- 一个有意思的问题：这个文法能否被 LL(1) 直接处理？如何处理？

$$S \rightarrow a S a$$

$$S \rightarrow a a$$

- 如果把这个问题改写，提取公因子“a”，那么我们可以得到：

$$S \rightarrow a X$$

$$X \rightarrow S a \mid a$$

那么以上文法能否被LL（1）处理？如果不能，那么能否对以上文法进行改写，使得改写之后，可以被LL（1）处理

自顶向下分析技术

- 自顶向下语法分析是编译原理中的核心内容，其中LL算法因其直观性和可预测性而被广泛应用
- LL(1)作为最基础的LL算法，通过向前查看一个符号来决定产生式的选择，但其局限性也十分明显——无法处理许多常见的语法结构，如左递归和需要更多前瞻符号的情况

自顶向下分析技术

- 自顶向下语法分析是编译原理中的核心内容，其中LL算法因其直观性和可预测性而被广泛应用
- LL(1)作为最基础的LL算法，通过向前查看一个符号来决定产生式的选择，但其局限性也十分明显——无法处理许多常见的语法结构，如左递归和需要更多前瞻符号的情况
- 随着语言复杂性的增加，传统LL(1)算法显得力不从心，这促进了LL(k)算法的发展。LL(k)通过向前查看k个符号来增强解析能力，但随着k值增大，解析表的复杂度会呈指数级增长，导致内存消耗过大和效率下降。这一局限性促使研究者们寻找更高效的解决方案

自顶向下分析技术

- 自顶向下分析技术前沿 -- LL(*) 算法

- 设计动机：传统LL(k)的困境

- LL(1)/LL(k)只能用固定k个符号做预测分支

- 现实语法常常“需要向前看任意多”，比如嵌套、前缀/后缀操作等

- LL(k)算法随着k增大，分析表规模呈指数增长，不可行

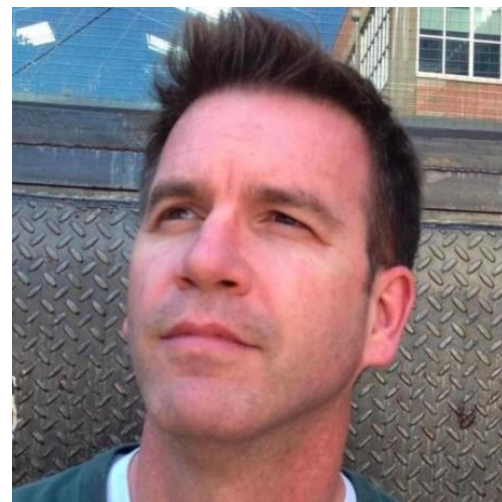
- LL(*)算法突破点：不再限定k，而是用有限自动机（DFA）识别任意长度的前缀来预测分支

- LL(1)/LL(k)必须通过人工“公因子提取”来消除预测冲突，文法可读性差，维护成本高

- LL(*)自动机能自动“看到底”，只要分支最终能区分，原始文法可以直接用，无需提取公因子

- 利用DFA的高效前缀判别能力，自动化构造预测器

- 由Terrence Parr教授创造，最初在ANTLR 3解析器生成器中实现



自顶向下分析技术

- 自顶向下分析技术前沿 -- LL(*) 算法

- 核心机制:

- LL(*) 算法的根本创新在于它采用了一种**基于正则表达式的超前查看模式**

- 传统LL(k)只能固定地查看前k个符号，而LL()则能够**智能地跨越重复结构**（如Kleene闭包）来获取所需的前瞻信息

自顶向下分析技术

• 自顶向下分析技术前沿 -- LL(*) 算法

➤ 核心机制：

□ ATN (Augmented Transition Network): 把语法规则转换为一种带 ϵ 转移、带规则调用/返回的 NFA 风格的图结构 (ANTLR 实现用 ATN 表示文法)

- 配置 (configuration): 一个 ATN 状态 + 解析上下文 (prediction context, 用于记录调用栈/规则嵌套信息)
- 配置集 (configuration set): 在某一步前瞻时可能的多个配置集合 (类似 NFA 的状态集合)
- DFA 状态: 若干 ATN 配置的集合 (即对 ATN 的幂集构造), 用于快速决策; 在 LL(*) 中 DFA 是按需构造并缓存的

□ SLL vs LL:

- SLL (Simple LL): 用空/简单预测上下文 (不考虑完整调用栈) 来快速预测
- LL (full LL): 使用完整的上下文信息 (prediction context) 进行精确预测。ANTLR 的 predictor 常先用 SLL (快), 若冲突则回退到 LL (慢但精确)。
- 语义谓词 (semantic predicates): 在候选路径上使用用户/语义检查表达式来剪枝 (在 DFA 构造或尝试匹配期间被评估)

自顶向下分析技术

• 自顶向下分析技术前沿 -- LL(*) 算法

➤ 核心机制:

□ LL(*) 算法的根本创新在于它采用了一种**基于正则表达式的超前查看模式**

□ 传统LL(k)只能固定地查看前k个符号，而LL(*)则能够**智能地跨越重复结构**（如Kleene闭包）来获取所需的前瞻信息

特性	LL(1)	LL(k)	LL(*)
前瞻能力	固定1个符号	固定k个符号	可变长度，理论上无限
处理左递归	不能处理	不能处理	间接处理（通过语法转换）
处理闭包能力	有限	有限	强（能处理Kleene星号）
解析表大小	小	随k值指数增长	相对紧凑（使用DFA）
内存消耗	低	中到高	中等
适用语法	简单语法	中等复杂度语法	复杂语法

自顶向下分析技术

- 自顶向下分析技术前沿 -- LL(*) 算法

- 核心机制:

- LL(*) 算法的根本创新在于它采用了一种**基于正则表达式的超前查看模式**

- 传统LL(k)只能固定地查看前k个符号，而LL()则能够**智能地跨越重复结构**（如Kleene闭包）来获取所需的前瞻信息

- LL(*)在保持相对紧凑的内存使用的同时，提供了更强大的解析能力

自顶向下分析技术

- 自顶向下分析技术前沿 -- LL(*) 算法

- 核心机制:

- LL(1) 用 1 个前瞻符号做决定, 简单但能力受限; LL(k) 用固定 k, 但有时需要任意长前瞻
 - 现实语言 (含关键字/标识符歧义、复杂嵌套、泛型/比较符号冲突) 要求更强的预测能力
 - LL(*) 的思想: 允许任意长度的前瞻, 但用工程上可行的方法保证效率 —— 把“无限前瞻”降为对前缀的 DFA 匹配, 并在运行时按需构造/缓存该 DFA

自顶向下分析技术

- LL(*) 算法分析框架

- (1) 构建决策NFA

- 对于某规则的各分支，将语法片段转化为NFA

- NFA的路径代表不同分支所有可能前缀

- (2) NFA $\rightarrow$ DFA（子集构造法）

- 用标准的子集构造法将NFA转为DFA

- DFA的每个状态代表NFA状态的集合，终态指明命中哪个分支

- (3) 运行时预测

- 解析时，在决策点，将输入流交给DFA

- DFA根据输入流移动，直至进入某分支的终态，确定选择

- 如果DFA无法确定分支（例如前缀不唯一），可回溯尝试其它方案

自顶向下分析技术

- LL(*) 算法分析框架

- (1) 构建决策NFA

- 对于某规则的各分支，将语法片段转化为NFA

- NFA的路径代表不同分支所有可能前缀

- (2) NFA $\rightarrow$ DFA（子集构造法）

- 用标准的子集构造法将NFA转为DFA

- DFA的每个状态代表NFA状态的集合，终态指明命中哪个分支

- (3) 运行时预测

- 解析时，在决策点，将输入流交给DFA

- DFA根据输入流移动，直至进入某分支的终态，确定选择

- 如果DFA无法确定分支（例如前缀不唯一），可回溯尝试其它方案

自顶向下分析技术

- LL(*) 算法分析框架

- (1) 构建决策NFA

- 对于某规则的各分支，将文法片段转化为NFA

- NFA的路径代表不同分支所有可能前缀

- (2) NFA $\rightarrow$ DFA（子集构造法）

- 用标准的子集构造法将NFA转为DFA

- DFA的每个状态代表NFA状态的集合，终态指明命中哪个分支

- (3) 运行时预测

- 解析时，在决策点，将输入流交给DFA

- DFA根据输入流移动，直至进入某分支的终态，确定选择

- 如果DFA无法确定分支（例如前缀不唯一），可回溯尝试其它方案

自顶向下分析技术

• LL(*) 算法分析框架的例子

```
grammar LLStarDemo;
stmt
  : ID '=' expr      #assign
  | ID '(' argList? ')' #call
  | ID               #idref
  ;
```

```
expr : ID | INT ;
argList: expr (',' expr)* ;
```

```
ID : [a-zA-Z]+ ;
INT: [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

说明：三条替代都以 ID 开头；
区分需要看第二个符号：=/(紧
接 EOF (idref)。这是 LL(2)
可解决的情况，但我们用 LL(*)
展示 DFA 预测过程。

因为LL(*)被广泛应用于 ANTLR，我们在这个地方就用
ANTLR的G4 文法定义来表达

自顶向下分析技术

- LL(*) 算法分析框架的例子

- 输入示例 1: `foo = 42`

- Token 流: `ID('foo'), '=', INT('42'), $`

- 决策点: 在解析 `stmt` 时, 预测器从当前 token (`ID`) 起对每个 alternative 的路径进行并行探索:

- alt1 路径 (assign): 要匹配 `ID then '=' ...`

- alt2 路径 (call): `ID then '(' ...`

- alt3 路径 (idref): `ID then (alternative ends) — accept after ID (但可能还需检查下一符号是否允许)`

```
grammar LLStarDemo;  
stmt  
  : ID '=' expr      #assign  
  | ID '(' argList? ')' #call  
  | ID               #idref  
  ;
```

```
expr : ID | INT ;  
argList: expr (',' expr)* ;
```

```
ID : [a-zA-Z]+ ;
```

```
INT: [0-9]+ ;
```

```
WS : [ \t\r\n]+ -> skip ;
```

自顶向下分析技术

- LL(*) 算法分析框架的例子

- 输入示例 1: `foo = 42`

- 模拟步骤:

- 初始配置集包含三条路径都位于 ID 前（因为三条均以 ID 开头）。
 - 消费第一个 token ID（匹配所有路径的 ID）。现在配置集表示下一符号所期待的集合：{ '=', '(', EOF }（分别对应三个 alt）
 - 看下一个 token '=' → DFA 根据 '=' 边移动到只包含 alt1 的 accepting 状态 → 立即选择 alt1（assign）

```
grammar LLStarDemo;  
stmt  
  : ID '=' expr      #assign  
  | ID '(' argList? ')' #call  
  | ID               #idref  
  ;
```

```
expr : ID | INT ;  
argList: expr (',' expr)* ;
```

```
ID : [a-zA-Z]+ ;  
INT: [0-9]+ ;  
WS : [ \t\r\n]+ -> skip ;
```

结论：LL(*) 在很少的前瞻下就能确定替代（构造的 DFA 很小）。

自顶向下分析技术

- LL(*) 算法分析框架的例子

- 输入示例 2: `foo(x, y)`

- Token 流: `ID('foo'), '(', ID('x'), ',', ID('y'), ')', '$`
- 决策点: 在解析 `stmt` 时, 预测器从当前 token (`ID`) 起对每个 alternative 的路径进行并行探索:
 - alt1 路径 (assign): 要匹配 `ID then '=' ...`
 - alt2 路径 (call): `ID then '(' ...`
 - alt3 路径 (idref): `ID then (alternative ends) — accept after ID (但可能还需检查下一符号是否允许)`
- 第二符号 `'('` 导致 DFA 移向只含 alt2 的接受状态 → 选择 call

```
grammar LLStarDemo;
stmt
    : ID '=' expr          #assign
    | ID '(' argList? ')'  #call
    | ID                   #idref
    ;
```

```
expr : ID | INT ;
argList: expr (',' expr)* ;
```

```
ID : [a-zA-Z]+ ;
INT: [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

第二章 语法分析

(自顶向下的分析技术)

冯洋

南京大学计算机学院

fengyang@nju.edu.cn